Programar con IA

Taller de 2 horas · OpenCode, Copilot o Cursor

¿Qué es programar con IA?

- **Autocompletado:** sugiere la siguiente línea
- **Chat:** responde preguntas sobre el código
- **Agente:** recibe un objetivo, lee, edita y ejecuta
- **OpenCode** está en el extremo agente

El bucle

- Intención → Contexto → Instrucción → Propuesta (diff) → Verificación

La IA abarata escribir y explorar. No abarata decidir ni verificar.

Los 8 elementos

1 Modelo

2 Contexto

3 Instrucción

4 Modos

5 Herramientas y permisos

6 Verificación

7 Reglas

8 Hooks

Modelo

- Predice tokens, no “conoce” tu repo
- La ventana de contexto es lo que cabe en la conversación
- Por eso inventa APIs plausibles
- **Demo:** /models

Contexto

- @archivo mete el contenido real en la conversación
- @archivo#20-45 acota un rango de líneas
- Sin contexto, el agente adivina
- **Demo:** @gastos.py y @tests/test_gastos.py

Instrucción

- **Vago:** “Arregla los tests”
- **Preciso:** objetivo + formato + límites + criterio de aceptación

Un prompt preciso es un contrato verificable.

Modos

- **Plan:** explora y propone, no edita
- **Build:** edita y ejecuta
- Shift+Tab cambia de agente

Primero pensar, después ejecutar.

Herramientas y permisos

- Lee, busca, edita y ejecuta comandos
- Los permisos deciden qué se aprueba
- En opencode.jsonc: shell pide aprobación, los tests no

Aprobar no es confiar: es revisar.

Verificación

- La respuesta segura no es evidencia
- Tests, tipos y lectura del diff
- !python3 -m unittest discover -s tests -t .

La IA propone; tú verificas.

Reglas

- Reglas permanentes del proyecto en AGENTS.md
- OpenCode lo lee automáticamente
- Es un prompt versionado
- Ejemplo: idioma, docstrings, sin dependencias

Hooks

- Interceptan operaciones vivas del flujo
- Son API de plugin, en TypeScript
- .opencode/plugins/ se carga solo
- Ejemplo: redactar un secreto antes de enviarlo

Reglas vs. hooks

- **Reglas:** Markdown, las interpreta el modelo
- **Hooks:** TypeScript, los ejecuta OpenCode
- Regla = probabilidad; hook = garantía

Las reglas piden; los hooks garantizan.

Dónde vive cada extensión

- agents/ → <nombre>.md, se carga al iniciar
- commands/ → <nombre>.md se invoca como /nombre
- skills/ → <id>/SKILL.md, se carga cuando el agente la invoca
- plugins/ → index.ts, se carga al iniciar

Todo vive dentro de .opencode/.

Agente, regla y comando

	Agente	Regla	Comando
Archivo	agents/x.md	AGENTS.md	commands/x.md
Frontmatter	sí, mode	no tiene	opcional
Cómo entra	Shift+Tab o subagente	siempre	/nombre
Qué cambia	quién responde	qué sabe del repo	cómo pides

Mismo formato, distinto disparador: uno cambia quién, otro qué sabe, otro cómo pides.

Lo que no tiene carpeta

- **Reglas:** AGENTS.md en la raíz, no una carpeta
- **Hooks:** código dentro de plugins/
- El cargador no adivina: cada ruta tiene un formato
- Todo .ts en plugins/ se carga solo

Si el archivo no está en la ruta correcta, no existe.

Las cuatro en acción

- **Agente:** revisor → “usa el subagente revisor”
- **Comando:** /explicar gastos.py
- **Skill:** @correr-tests
- **Plugin:** escribe SECRETO y observa

Un agente cambia quién responde; un comando, cómo pides; una skill, cómo trabaja; un plugin, qué pasa siempre.

Demo

- gastos.py con 6 pruebas
- **Estado inicial:** 2 pasan, 4 fallan
- Un ciclo completo: del rojo al verde

Práctica: 5 misiones

- **1 Contexto** — @
- **2 Instrucción y verificación** — vago vs. preciso
- **3 Modos** — Plan y Build
- **4 Reglas** — AGENTS.md
- **5 Opcional** — permisos y alucinación

Riesgos

- **Alucinación:** APIs que no existen
- **Secretos:** nunca pegues tokens en el chat
- **Aceptar sin leer:** revisa el diff
- **Dependencias inventadas:** respeta AGENTS.md
- **Licencias:** verifica el origen

Qué te llevas

- El bucle de 5 pasos
- Los 8 elementos
- Un repo que funciona
- Reglas piden; hooks garantizan
- **Checklist:** contexto → instrucción → plan → ejecutar → verificar

Cierre

La IA no reemplaza leer el código. Lo hace más rápido cuando le das contexto y le exiges evidencia.

Gracias